

How-To: Switchable Execution Backends and Sandboxing

ragent can run a session's tool invocations in one of four **execution backends**: on the host (`local`, the default), inside a container sandbox (`docker` or `podman`), or against a second ragent server reached over its REST+SSE API (`remote`). Selecting a backend changes *where* a tool runs - it never changes the agent loop, the LLM providers, the tool registry, the permission system, or the event bus (spec `openhands` FR-001, assumption A1).

This guide covers four things:

- The `/backend` TUI command that lists, inspects, and switches the active backend.
- How the switchable backend subsystem works end to end - the trait, resolution, the registry, the per-session override, and the “never fall back to host” rule.
- The configuration that declares backends.
- A complete worked example: building an image, storing a credential, and running a session inside a Podman (or Docker) sandbox.

Scope. This guide covers backend selection and sandboxing. For the static permission pipeline and the 7-layer bash model see `permissions.md`. For the config key reference see `config.md` section 7.36. For the `/backend` command quick reference see `slashcommands/backend.md`. For the requirements see `specs/openhands/SPEC.md` FR-001 through FR-037.

Table of Contents

- 1. Concepts
 - 2. The `/backend` Command
 - 2.1 Syntax
 - 2.2 The switcher panel
 - 2.3 Health states
 - 2.4 Persistence
 - 2.5 Output
 - 3. How the Switchable Backend Works
 - 3.1 The `ExecutionBackend` trait
 - 3.2 Resolution on every dispatch
 - 3.3 The backend registry
 - 3.4 The per-session override
 - 3.5 The container backend
 - 3.6 The remote backend
 - 3.7 Credential injection
 - 3.8 No fallback to host
 - 4. Configuration Reference
 - 5. Worked Example: a Docker/Podman Sandbox
 - 6. Troubleshooting
 - 7. See Also
-

1. Concepts

Term	Meaning
Execution backend	The layer that decides where one tool call executes. Selected by the <code>execution_backend</code> config key plus optional backends entries.
Kind	One of <code>local</code> , <code>docker</code> , <code>podman</code> , <code>remote</code> (<code>ExecutionBackendKind</code> , <code>crates/ragent-config/src/config.rs</code>).
Descriptor	A <code>BackendConfig</code> entry: a stable id, a display name, a kind, kind-specific connection fields (<code>image</code> , <code>workspace</code> , <code>url</code> , <code>api_key</code>), and a list of credential <i>names</i> .
Registry entry	The read model over a descriptor: id, name, kind, a secret-free <code>ConnectionDescriptor</code> , and a health state (FR-004).
Sandbox	The long-lived container a <code>docker/podman</code> backend provisions for a session and dispatches every tool call into.

The important invariants (FR-001, FR-002, FR-037): backend selection is confined to the *tool-execution leaf*. The 7-layer bash validator, the file-path containment guard, the permission system, and the always-allowed codeindex hardwiring all run **before** any backend dispatch and are unchanged by the choice.

2. The `/backend` Command

`/backend` is the TUI surface for the execution backend (FR-008). It is implemented in `crates/ragent-tui/src/app/slash`. (`handle_backend_command`, dispatch arm "backend") and `crates/ragent-tui/src/app/backend_panel.rs`. It is TUI-only - there is no `ragent backend` CLI subcommand.

2.1 Syntax

```

/backend          # open the switcher panel
/backend show     # alias of /backend
/backend list     # alias of /backend
/backend local    # switch to the local host backend
/backend docker   # switch to a Docker sandbox backend
/backend podman   # switch to a Podman sandbox backend
/backend remote   # switch to a remote backend
/backend <id>     # switch to a backend by its registered id
/backend help     # usage help

```

Form	Description
<code>/backend</code>	Open the switcher panel: the active backend and every registered backend's health.
<code>/backend show /</code> <code>/backend list</code>	Alias of <code>/backend</code> .
<code>/backend <kind></code>	Switch directly to a backend <i>kind</i> (<code>local</code> , <code>docker</code> , <code>podman</code> , <code>remote</code>). Persists the bare kind label.

Form	Description
<code>/backend <id></code>	Switch directly to a backend by its registered <i>id</i> (a backends entry, or an inline <code>execution_backend</code> descriptor).
<code>/backend help</code>	Print the usage table. Read-only; changes no state.

The selector is matched case-insensitively against a row's `id` **or** its `kind` (`switch_backend_by_selector`, `backend_panel.rs`).

2.2 The switcher panel

With no argument the command opens a **modal switcher panel** titled `Execution backend - active: <kind>`. Each row carries the backend's `id`, `kind`, `health`, and a secret-free connection summary; the active row carries an `[active]` marker.

Key	Action
Up / Down	Move the block cursor between rows.
Enter	Switch the active backend to the highlighted row.
Esc	Close the panel.

While the panel is open it owns the keyboard and the message input field stays locked. A row whose health is not ok is dimmed and **cannot** be selected: the switch is refused and the active backend is left unchanged, so the switcher can never leave the session on a backend that cannot run a tool. The rows are a snapshot taken when the panel opens (`BackendPanelState::build`); the panel does not re-probe health on each frame.

2.3 Health states

Health is computed from the config and the host, with a live probe only for remote entries (`initial_health` / `refresh_health` in `crates/ragent-agent/src/backend/registry.rs`).

Kind	ok when	unavailable when
<code>local</code>	always - the host is always available (FR-019)	never
<code>docker</code> / <code>podman</code>	the runtime binary resolves on PATH (FR-026)	the binary is missing (the detail names it)
<code>remote</code>	a base URL and a bearer key are configured (FR-030)	either is absent; a live GET <code>/health</code> probe refines the state (no key is sent)

2.4 Persistence

A successful switch writes `execution_backend: "<kind>"` into the loaded config file - the project-local `.ragent/ragent.json` when one was loaded, otherwise the global config (`persist_active_backend` / `backend_config_path`) - using an atomic temp-file-then-rename update. The per-turn config cache is then invalidated, so the change takes effect on the **next** turn. After a switch the status bar shows `B:<kind>` (a non-local backend is highlighted).

Because the persistence is by **kind label**, a descriptor-free switch such as `/backend docker` writes the bare label. To carry an image, workspace, URL, or key, declare a full descriptor in the backends array (or as an inline `execution_backend` object) and select it with `/backend <id>`.

2.5 Output

- A switch appends a `From: /backend` message bubble and sets the status to `backend: <kind>`.
- A refusal appends a `[warn]` message explaining why and sets the status to `backend: unavailable / backend: unknown`.
- An unregistered id is refused with a pointer to `/backend`.
- No permission prompt is raised and no LLM call is made.

3. How the Switchable Backend Works

All of the machinery lives in `crates/ragent-agent/src/backend/`.

File	Responsibility
<code>mod.rs</code>	The <code>ExecutionBackend</code> trait, <code>BackendError</code> , the adapters, and resolution/caching helpers.
<code>registry.rs</code>	The durable backend registry (FR-004) and remote registration (FR-030).
<code>detection.rs</code>	Container-runtime detection on PATH, with podman as the default (FR-026).
<code>container.rs</code>	The docker/podman sandbox backend (FR-020, FR-031, FR-035).
<code>remote.rs</code>	The remote REST+SSE backend (FR-021, FR-034).
<code>secrets.rs</code>	Spawn-time credential resolution and injection (FR-010).

3.1 The ExecutionBackend trait

```
pub trait ExecutionBackend: Send + Sync {
    fn kind(&self) -> ExecutionBackendKind;
    async fn execute_tool(&self, call: BackendToolCall<'_>) -> anyhow::Result<ToolOutput>;
}
```

`BackendToolCall` carries the `Tool` handle, its JSON input, and the `ToolContext` (working directory, config, storage). Four implementations ship:

- `LocalBackend` - runs the tool on the host, exactly as before.
- `ContainerBackend` - provisions a container and dispatches into it.
- `RemoteBackend` - relays the turn to a second server.
- `PendingBackend` - a placeholder that fails every call, so a non-local kind with no adapter fails rather than silently running on the host (FR-031).

3.2 Resolution on every dispatch

Each tool dispatch resolves the backend from the session's live config (`dispatch_tool_leaf` in `crates/ragent-agent/src/session/processor.rs` calls `resolve_backend_with_secrets`):

```
pub fn resolve_backend_with_secrets(
    config: &Config,
```

```

    working_dir: &std::path::Path,
    storage: Option<Arc<crate::storage::Storage>>,
) -> Arc<dyn ExecutionBackend>

• local -> local_backend().
• docker / podman -> a ContainerBackend built from the effective descriptor, wired to the encrypted credential store.
• remote -> a RemoteBackend.
• An absent or unknown execution_backend key resolves to local (Config::effective_execution_backend, FR-019); a typo is flagged by Config::has_unknown_execution_backend rather than mis-resolving onto a more capable backend.

```

Resolved backends are cached per (kind, descriptor, working dir, credential set) so a session reuses one instance.

3.3 The backend registry

BackendRegistry::from_config builds the read model from the config (FR-004):

1. The built-in local entry, id local, display name Local (host), always present and healthy (FR-019).
2. One entry per backends descriptor, deduplicated by id.
3. The entry the execution_backend key resolves to (so an inline descriptor is visible).
4. A synthesised entry for a bare kind label with no matching descriptor (for example execution_backend: "podman"), so the active backend is always visible.

Each entry exposes a ConnectionDescriptor that carries credential **names** and presence flags only - never a value (FR-010, FR-035). For a container the summary is of the form podman container=<name> image= workspace=/projects volume=<vol>.

3.4 The per-session override

Automation runs (spec openhands T-016, FR-033) need to confine a run to a specific backend regardless of the ambient config. This is done with a process-global map in crates/ragent-agent/src/automation/mod.rs:

```

pub fn register_session_backend(session_id: &str, descriptor: BackendConfig);
pub fn clear_session_backend(session_id: &str);
pub fn session_backend_override(session_id: &str) -> Option<BackendConfig>;
pub fn apply_session_backend_override(config: &Arc<Config>, session_id: &str) -> Arc<Config>;

```

apply_session_backend_override is applied once per turn in crates/ragent-agent/src/session/loop_steps.rs (before hooks are parsed), so the whole turn - and every tool dispatch in it - is confined to the run's configured backend. A run with no override returns the shared Arc unchanged, so the common path stays allocation-free; a run with an override gets a modified clone whose execution_backend names the selected descriptor. A relay failure therefore never silently re-runs the turn on the host (FR-031).

3.5 The container backend

ContainerBackend (container.rs) answers the “where” half of a tool invocation:

- **Provisioning (FR-009, FR-020).** On the first dispatch it starts **one long-lived container** for the session:

```
<runtime> run -d --name ragent-sandbox-<hash> \
    -v ragent-sandbox-vol-<hash>:/projects -w /projects \
    -e NAME=VALUE ... \
    <image> sleep infinity
```

The image must provide a minimal POSIX shell userland (sh, sed, grep, find, base64); the container command is `sleep infinity`, so the sandbox stays up for the whole session and each tool call is one cheap exec. The container is created before the first tool call, so the project is made available to it before any dispatch (FR-009). A default image is required - if the descriptor names none, provisioning fails.

- **Workspace isolation (FR-020, FR-035).** The workspace is a **container-named volume** (`ragent-sandbox-vol-<hash>`), never a host bind mount. A tool inside the container cannot read or write a host path - it is the strongest form of “shall not expose the host working directory”. The container and volume names are derived deterministically from (`runtime`, `id`, `image`, `workspace`, `credential` names, `host working dir`), so a restarted process reuses the same sandbox instead of leaking a new one, while a changed credential set gets a freshly-provisioned container (FR-010, FR-023).
- **Dispatch.** Only the shell and file/search tools are served:
 - `bash` - validated by the **7-layer host validator** (`ragent_tools_core::bash::validate_shell_command`) before it is executed, so the sandbox can never be used to bypass the security model (FR-002, FR-037); it then runs as `exec <name> sh -lc '<command>'` with the workspace as the working directory.
 - `read`, `write`, `create`, `append_file`, `rm`, `mkdir`, `list`, `glob`, `grep` - translated to a sandbox `exec` script (content is transported base64-encoded so it cannot be mangled by quoting) and confined to the workspace mount point.
 - **Every other tool is refused** - the container backend has no adapter for it and never falls back to host execution (FR-031). This means that while a container backend is active, tools outside that subset (for example `webfetch`, `codeindex_*`, `mf_*`) are refused for the turn rather than routed.
- **The workspace field.** A `BackendConfig.workspace` that is an **absolute path** overrides the container-side mount point; a relative value such as `"."` keeps the default `/projects`.

3.6 The remote backend

`RemoteBackend` (`remote.rs`) points at a second ragent server (usually on another host) and runs the session **against that server**. It does not execute tools locally; it relays the whole turn over REST+SSE and mirrors that server’s event stream into the local event bus (FR-021):

```
POST <url>/sessions { "directory": "<dir>" } -> { "id": ... }
POST <url>/sessions/<id>/messages { "content": "<prompt>" } -> text/event-stream
POST <url>/sessions/<id>/permission/<req_id> { "decision": "allow" }
```

The `send_message` route both starts the turn and streams its events, so one request drives the whole turn (`relay_turn`). Frames are decoded into `RemoteUpdates` and re-published onto the **local** bus under the local session id (`crates/ragent-agent/src/session/remote_dispatch.rs`), so the TUI renders a remote turn exactly like a local one. The remote session is cached per local session id, so a follow-up message reuses the same remote session.

Unreachability (FR-034):

- A server that cannot be reached before the turn starts is a provisioning failure (`RemoteErrorKind::Unreachable`) and fails the turn with no local execution (FR-031).
- A stream that drops **part-way through** a turn is `RemoteErrorKind::Protocol` and likewise never re-runs the turn's tools locally (FR-034).
- The relay is bounded by `TURN_TIMEOUT` (15 minutes) and polls a cancellation flag, so an unresponsive server cannot hang the session.

A remote descriptor with no url fails at provisioning rather than falling back to host execution (`validate_remote_backend`).

3.7 Credential injection

A docker/podman/remote descriptor may list credentials - **names**, never values. At spawn time the backend resolves each name from the encrypted credential store (the same `provider_auth` table the `ragent auth` command writes to) and injects it (FR-010):

```
pub fn resolve_descriptor_secrets(
    descriptor: &BackendConfig,
    resolver: &dyn SecretResolver,
) -> anyhow::Result<Vec<ResolvedSecret>>;

pub fn container_env_args(secrets: &[ResolvedSecret]) -> Vec<String>; // ["-e", "NAME=VALUE", ...]
```

- Values are injected as container environment variables (`-e NAME=VALUE`), never written into the image, the workspace, or `ragent.json` (FR-035).
- A name the store does not hold is a **hard provisioning failure** - the sandbox is never started with a required credential silently missing.
- Every resolved value is registered with the shared redaction registry, so any later rendering (a log line, a `/config` dump, an error echoed back by the runtime) masks it.
- A running container is reused only if it already carries every credential the descriptor names; a changed credential set re-provisions the container with a fresh environment while re-mounting the same named volume, so the conversation's files survive (FR-023) and secret rotation takes effect at the next spawn.

3.8 No fallback to host

The single rule that ties the subsystem together (FR-031): if a non-local backend fails to provision, the turn fails with a `BackendError` whose `BackendErrorKind` is `Provision` (or `Protocol` for a mid-turn remote drop). The provisioning error is **sticky** - it is replayed on every subsequent dispatch in that session - and the tool is **never** run on the host instead. The error text always names the backend and states plainly "the tool was not executed on the host".

4. Configuration Reference

Two keys drive backend selection (see `config.md` section 7.36 for the full field table).

```
{
    // Terse form: select a kind by label.
    "execution_backend": "podman",
}
```

```
// Registry form: declare backends and select one by id (or by kind label).
"backends": [
  { "id": "sandbox", "name": "Podman sandbox", "kind": "podman",
    "image": "ghcr.io/yourorg/ragent-sandbox:latest", "workspace": "/projects",
    "credentials": ["EXAMPLE_API_KEY"] },
  { "id": "build-farm", "name": "Remote build host", "kind": "remote",
    "url": "http://10.0.0.5:9100", "api_key": "sk-ragent-remote" }
]
}
```

`execution_backend` accepts either a bare label (`local`, `docker`, `podman`, `remote`) or an inline descriptor object (`{ "id", "kind", "image", "workspace" }`). An object that names a registered id (with no kind) resolves its kind from the matching backends entry. An absent or unknown value resolves safely to `local` (FR-019).

BackendConfig fields:

Field	Type	Default	Description
<code>id</code>	<code>String</code>	<code>""</code>	Stable identifier used to select this backend.
<code>name</code>	<code>Option<String></code>	<code>None</code>	Human-readable display name; defaults to <code>id</code> .
<code>kind</code>	<code>String</code>	<code>""</code>	Backend kind: <code>local</code> , <code>docker</code> , <code>podman</code> , or <code>remote</code> .
<code>image</code>	<code>Option<String></code>	<code>None</code>	Container image reference (required for <code>docker/podman</code>).
<code>workspace</code>	<code>Option<String></code>	<code>None</code>	Container-side workspace mount point. An absolute path overrides the default <code>/projects</code> ; a relative value such as <code>."</code> keeps the default.
<code>url</code>	<code>Option<String></code>	<code>None</code>	Base URL of a remote server (<code>remote</code> kind).
<code>api_key</code>	<code>Option<String></code>	<code>None</code>	Bearer key for a remote server (<code>remote</code> kind). Redacted in diagnostics. A literal <code>api_key</code> takes precedence over a named credential.
<code>credentials</code>	<code>Vec<String></code>	<code>[]</code>	Names of encrypted-store credentials resolved at spawn time and injected as container environment variables (FR-010).

`podman` is the default container runtime when none is specified (FR-026).

5. Worked Example: a Docker/Podman Sandbox

This walks through running a session inside a Podman sandbox, end to end. Docker works identically - substitute `docker` for `podman` and `"kind": "docker"`.

Step 1 - Confirm a runtime is present (FR-026)

```
podman --version          # or: docker --version
```

ragent probes PATH for the runtime binary (it never spawns the runtime just to check). If podman is missing, the /backend row for a Podman backend reads unavailable and cannot be selected. Install podman (or docker) or stay on local.

Estimated time: 2 minutes if Podman is already installed, 5-10 minutes to install it fresh.

Step 2 - Build a sandbox image

The image must provide a minimal POSIX shell userland. A small Debian-slim base is enough (coreutils supplies base64, findutils supplies find):

Dockerfile:

```
FROM debian:bookworm-slim
RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        coreutils findutils grep sed ca-certificates \
    && rm -rf /var/lib/apt/lists/*
# The backend starts the container as `sleep infinity`, so no ENTRYPOINT/CMD
# is required; anything that keeps PID 1 alive works.
```

Build it with the runtime that will back the sandbox:

```
podman build -t ragent-sandbox:latest -f Dockerfile .
```

(The backend starts the container with `sleep infinity`, so the image does not need a long-running entrypoint.)

Step 3 - Store the credential the sandbox needs

A descriptor's credentials list names entries in the encrypted credential store - the same `provider_auth` table the `ragent auth` command writes to. Store the value under the name the descriptor will list (never put the value in `ragent.json`):

```
ragent auth EXAMPLE_API_KEY sk-example-0123456789
```

The value is stored encrypted and resolved by name **at spawn time**, then injected as the container environment variable `EXAMPLE_API_KEY`.

Step 4 - Declare the backend

Add the descriptor and select it. In the project-local `.ragent/ragent.json`:

```
{
  "execution_backend": "sandbox",
  "backends": [
    {
      "id": "sandbox",
      "name": "Podman sandbox",
      "kind": "podman",
      "image": "ragent-sandbox:latest",
    }
  ]
}
```

```

    "workspace": "/projects",
    "credentials": ["EXAMPLE_API_KEY"]
  }
]
}

```

Notes:

- workspace is the **container-side** mount point. "/projects" is the default; a relative value such as "." also keeps the default.
- credentials carries **names only** - EXAMPLE_API_KEY here, matching the ragent auth EXAMPLE_API_KEY ... above.
- Omit execution_backend (or set it to "local") to keep host execution.

Step 5 - Activate the backend

Start the TUI and check the switcher:

```
/backend
```

The panel lists local (active) and sandbox (kind podman, health ok). Move with Up/Down and press Enter on sandbox - or select it in one step:

```
/backend sandbox
```

Either way, ragent persists execution_backend: "sandbox" to the config file and the status bar now shows B:podman. The change applies to the **next** turn.

Step 6 - Run a turn and observe the sandbox

Ask the agent to inspect the workspace:

List the files in the workspace with ``bash: ls -la``.

What happens on the first tool call:

1. ragent provisions the sandbox container: `<runtime> run -d --name ragent-sandbox-<hash> -v ragent-sandbox-vol-<hash>:/projects -w /projects -e EXAMPLE_API_KEY=<value> ragent-sandbox:latest sleep infinity`.
2. The `ls -la` command is validated by the **7-layer bash validator** on the host as usual, then executed inside the container as `exec <name> sh -lc 'ls -la'`.

You will see that /projects starts **empty** - the host working directory is not mounted. A read or ls of a host path inside the sandbox finds nothing, and a file you create inside the sandbox does **not** appear on the host. That is FR-020/FR-035 working as designed.

If the model tries a tool outside the sandbox subset (for example webfetch or codeindex_search), it is refused for that turn - the container backend has no adapter for it and will not run it on the host (FR-031).

Step 7 - Fail-without-fallback (prove FR-031)

Point the descriptor at an image that does not exist:

```
"image": "ragent-sandbox:nonexistent"
```

The next tool call fails with a `Provision` error naming the image and stating the tool was not executed on the host. Fixing the image and re-running the turn provisions cleanly; the same named volume is re-mounted, so any files the conversation created earlier are still there.

Step 8 - Rotate a credential

Change the stored value:

```
ragent auth EXAMPLE_API_KEY sk-example-rotated
```

Because the credential set is part of the sandbox’s deterministic identity, the next spawn **replaces** the container with a fresh environment carrying the new value while re-mounting the same volume (FR-010, FR-023).

Step 9 - Return to the host and clean up

```
/backend local
```

The status bar returns to `B:local`. To reclaim disk used by the sandbox:

```
podman rm -f $(podman ps -a --filter name=ragent-sandbox --format '{{.Names}}')
podman volume rm $(podman volume ls --filter name=ragent-sandbox --format '{{.Name}}')
```

6. Troubleshooting

Symptom	Cause	Fix
<code>/backend row</code> for the sandbox is unavailable and cannot be selected	the runtime binary is not on PATH (FR-026)	install <code>podman/docker</code> , or select <code>local</code> .
Turn fails with “container runtime ... was not found on PATH”	the descriptor names a runtime that is absent	install it, or drop image/switch the descriptor’s kind.
Turn fails with “has no container image configured”	image is missing on the descriptor	set image.
Turn fails with “failed to start the ... sandbox container from image ...”	the image cannot be pulled/started	build or pull the image; the runtime’s own error is echoed (with secrets redacted).
Turn fails with “requires the credential ... which is not present”	the <code>credentials</code> name is not in the store	store it with <code>ragent auth <NAME> <VALUE></code> (FR-010).
A tool is refused with “no adapter”	the tool is outside the sandbox’s shell/file/search subset	use a tool in the subset, or switch to <code>local</code> (FR-031).
<code>/backend <id></code> reports “No registered backend matches”	the <code>id</code> is not declared in backends	add a descriptor, or use a kind label.

7. See Also

- `slashcommands/backend.md` - the `/backend` command quick reference.
- `config.md` section 7.36 - the `execution_backend` and `backends` config keys.
- `permissions.md` - the static permission pipeline and the 7-layer bash model that run before any backend dispatch.
- `llmsecurity.md` - the optional LLM security analyzer, a tightening-only layer over the same permission system.
- `specs/openhands/SPEC.md` - FR-001, FR-004, FR-008, FR-009, FR-010, FR-019, FR-020, FR-021, FR-023, FR-026, FR-030, FR-031, FR-033, FR-034, FR-035.